

Sleeping Machines: what is known

Computing in time with races, holds and vetoes · report, 27 September 2026

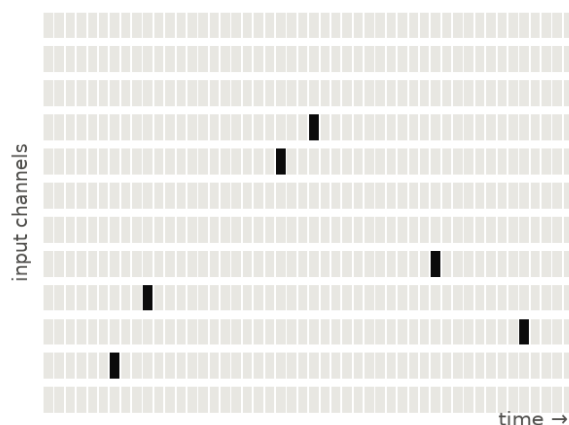
In plain terms

Today's neural networks are **clocked and dense**: at every step, every input is multiplied by every weight, whether or not anything happened. Many real signals are the opposite: long silences broken by precisely timed events (nerve spikes, trades on a market, sensor alarms), where *when* something happens is the information.

Sleeping Machines are networks that only work when an event arrives. A node waits. It fires when the right inputs arrive in the right time window ("B within 1.5 s after A"), and the first node to fire gives the answer, a *race*. Silence costs nothing, and time itself does the computing: a delay or a waiting window plays the role that a weight matrix plays in a dense network.

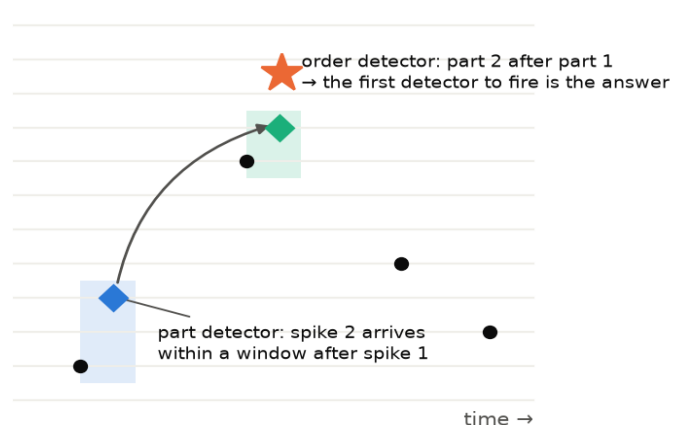
The questions are whether such networks can **learn** (without backpropagation or any dense machinery: a node adjusts only its few connections that were active, like moving money between accounts under a fixed budget) and whether they can **match or beat** MLPs and Transformers.

Clocked network: every cell, every tick



576 cells × every weight $\approx 10^5$ – 10^6 multiply-adds

Event network: work only when a spike arrives



≈ 10 events; silence costs nothing

Highlights

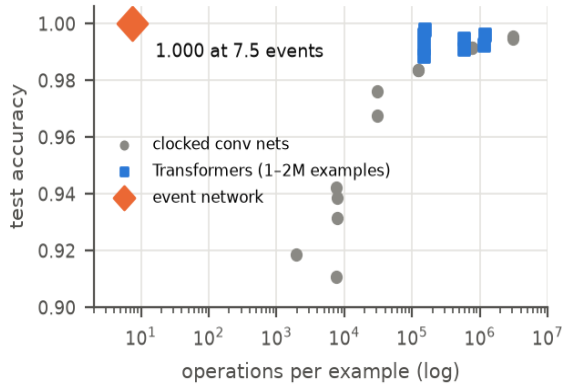
- **Same accuracy, 10,000–100,000× less computation.** On timing-pattern recognition a learned event network is perfect (1.000) using ≈ 7.5 events per example; Transformers reach 0.989–0.998 at 150k–1.2M multiply-adds after 1–2M training examples.
- **It groks where a Transformer does not.** Trained on 30% of all (a, b, c) triples, it learns (a + b + c) mod 17 and is 99.4–99.9% correct on the triples it never saw; a Transformer with weight decay stays at 3–63%.
- **Ten times less data.** On the depth-3 order task the event network is 99.8–99.9% correct after 1,000–2,000 examples seen once; a Transformer allowed as many passes as it likes needs about 10,000–20,000 for 99% (2,000: 33–41%; 5,000: 82–90%).
- **Deep order from a few thousand examples.** Recognizing which of 20 orders of four patterns occurred needs four levels of "this, then that". The network finds the right detectors among 55 million candidates and is 99.9–100% correct on 5 of 5 runs after 10–15k examples, with $\approx 2,000$ learning updates, ≈ 150 events per example and only $\approx 80k$ connections ever created. **At depth 4 a Transformer makes about ten times as many errors with the same 40k examples, and still 4–8 times as many with 2M** (0.990; 0.992–0.996 vs 0.999–1.000), at $\approx 7,000\times$ the computation; at depth 3 it matches only with 8–400× more data.
- **Composing parts: Transformer-level accuracy from one pass over the data, at $\approx 10^4\times$ less computation.** 15 classes built from ordered pairs of shared motifs: 0.990–0.999 (mean 0.9965) from 40k examples seen once, with learned timing windows, at ≈ 20 events per example; a Transformer

needs 2M examples for 0.9955–0.998 and reaches 0.9935–0.9965 given the same 40k examples 50 times.

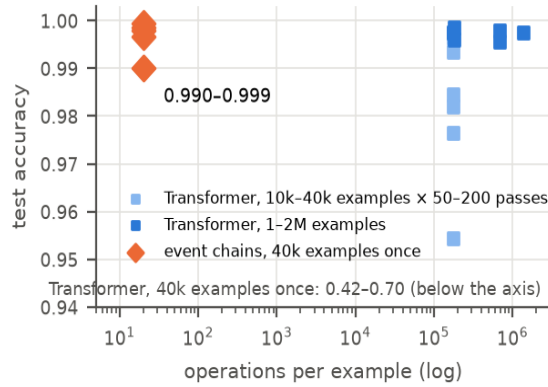
- **A world model of a real market stream within 0.07–0.18 nats of a Transformer at $\approx 1/3000$ of the computation.** On days it never saw, a small event network with slow regime counters beats a GRU point process and comes within 0.07–0.18 nats per event of a Transformer point process, at ≈ 40 operations per event instead of $\approx 110k$ – $130k$; the Transformer is the more accurate model.
- **Learning cost follows activity, not size.** Eight times more inputs ($12 \rightarrow 96$ channels) costs no more learning mistakes.
- **New theory, proved:** exactly what one event node can compute and where depth is needed; why a fixed weight budget lets a node learn an AND without knowing which half was wrong; why learning deep order needs a little exploration and a safety margin.

Scope of the evidence. The supremacy results (timing, composition, deep order, grokking) are on synthetic tasks built to test one capability at a time, where the target is exactly expressible by the primitives. The theory behind them (what a node computes, mistake bounds logarithmic in the candidate basis, cost proportional to events) is not task-specific: the reason to expect them to carry over to sparse, precisely timed real streams. On the real data tested so far the event network is competitive at a small fraction of the computation, not ahead: spoken digits 0.675 vs ≈ 0.70 (LSTM); a market world model 0.07–0.18 nats behind a Transformer point process; no trading edge after fees in four markets; on a real event-camera benchmark (DVS128 Gesture) far behind: 0.70 vs 94–98% published.

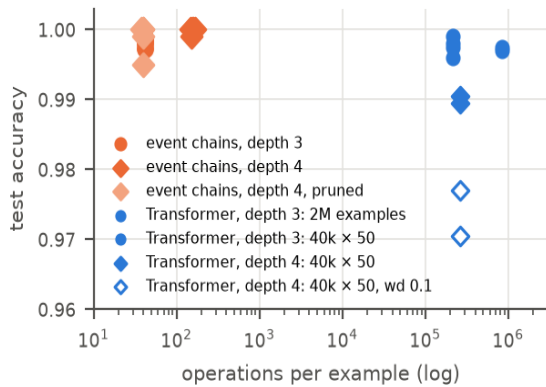
**Timing patterns:
same accuracy, $\sim 10^5 \times$ less work**



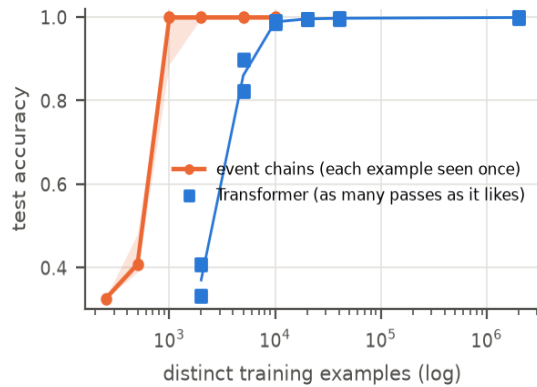
**Composing parts: Transformer-level accuracy
from one pass, $\sim 10^4 \times$ less work**



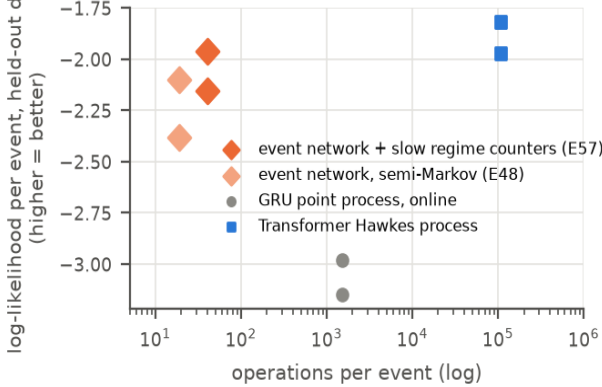
**Deep order: parity at depth 3,
 $\sim 10 \times$ fewer errors at depth 4 (equal data)**



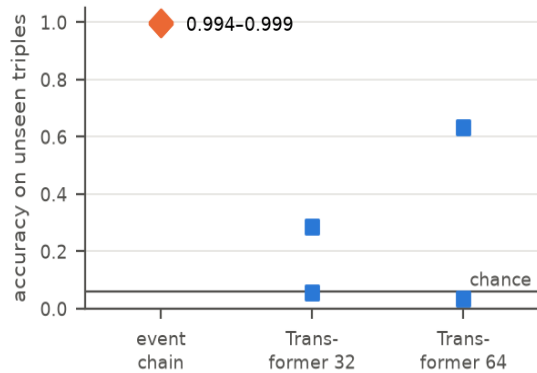
**Data efficiency (depth-3 order): 0.99 from
 ~ 1 -2k examples vs ~ 10 -20k for a Transformer**



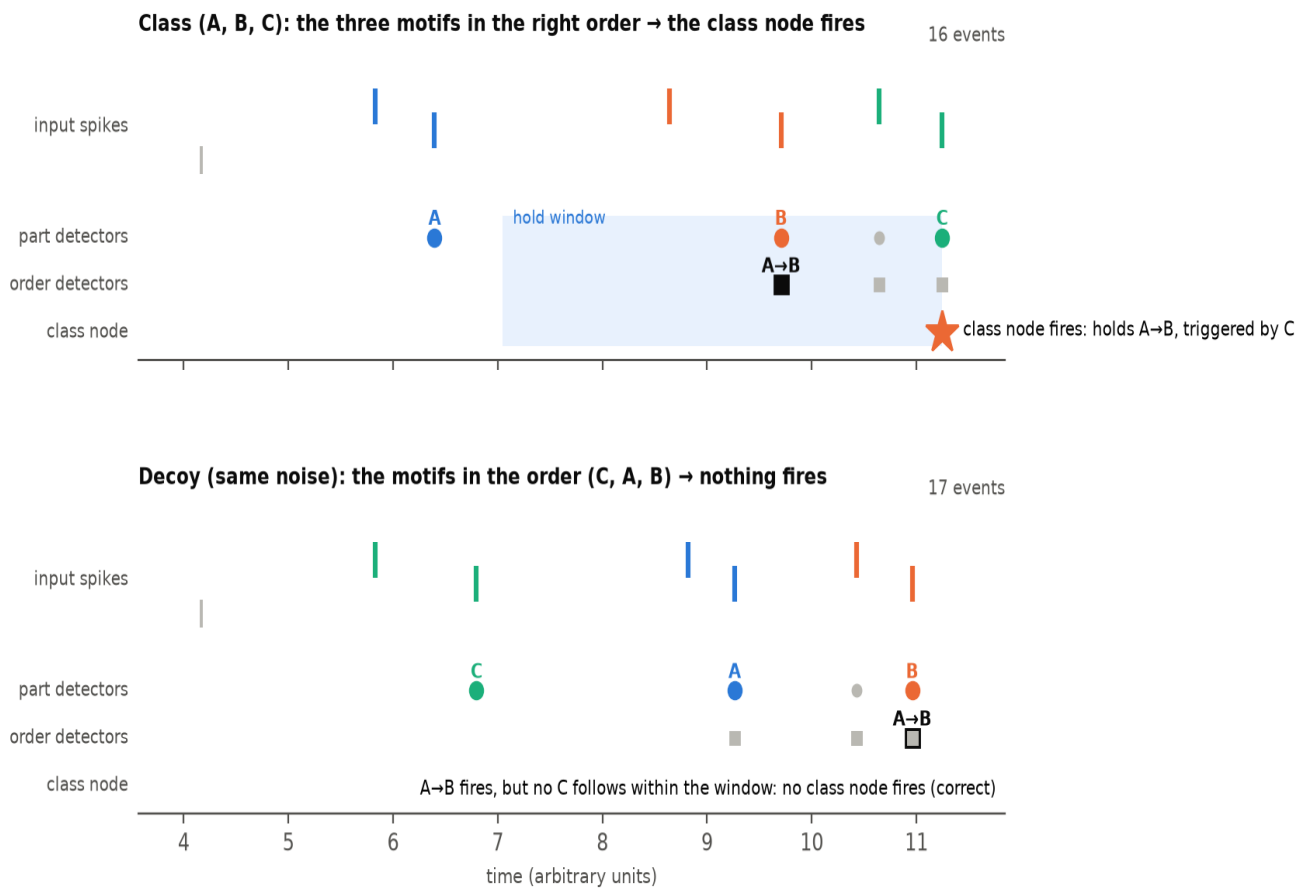
**Market world model: within 0.07-0.18 nats
of a Transformer, $\sim 1/3000$ of the work**



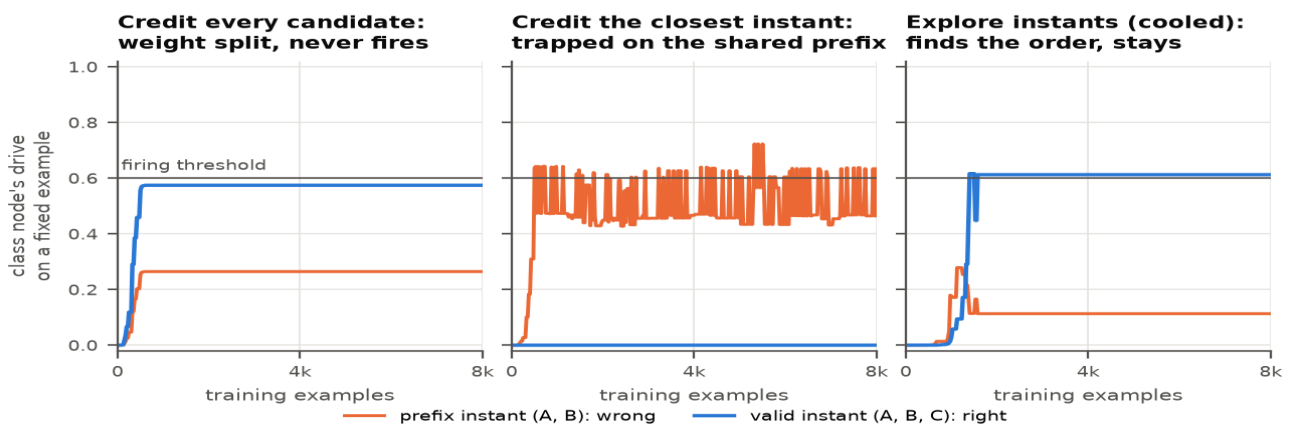
**Grokking (a + b + c) mod 17 from 30%:
the event chain generalizes**



What the network actually does on one example: spikes arrive; part detectors fire when two spikes are close enough in time; an order detector fires when part B follows part A; the class node holds that and fires when C arrives. With the same motifs in another order, the “A then B” detector still fires but nothing completes the pattern.



Why learning it is not trivial. When a detector for “A, then B, then C” fails to fire, which of its connections should change? Crediting every candidate spreads the weight so thinly that the node never fires; crediting the tempting shortcut (“A, then B” is shared with another class) traps it; exploring a little, then settling, finds the right order and keeps it (§84).



Where it does not win yet: event-camera gestures (0.70 vs 94–98% published), spoken digits (0.675 vs 0.70 for a published LSTM), and trading, where no learner beats buy-and-hold on this data (an audit shows why: the predictable edge, ≈ 1 bp per trade, is below any taker fee). **Next:** a path to generative language models built this way (section 10).

Sleeping Machines proposes that computation can happen **in time rather than memory**: candidate events race, the first to fire cancels the rest, and what a node computes is set by delays, by how long it holds an input, and by inhibition that arrives in time. This report states what is now known about such networks, why, and what remains open. Derivations and proofs are in experiments/THEORY.md (cited as §n).

Summary

- **What one node computes is exactly characterized, and it says where depth is needed.** A node with a trigger, hold and veto inputs accepts a product set in lag coordinates; one node orders at most three events; two layers compute every conjunction of bounded time differences, three layers every union (§71, proved; exhaustive search agrees). Nodes with state (arm/disarm) are strictly stronger.
- **Event networks learn timing with very few mistakes** if winning is positional (pull the event that should have won, never push a loser later) and order uses held intervals, not aligned delays: $O(\log N + \text{a few})$ mistakes per node.
- **On a timing task a learned event network matches or beats dense models at 10^4 – $10^5\times$ lower cost**, with nothing given: 1.000 at 7.5 synaptic events per episode vs 0.989–0.996 for event-token Transformers at 146k–1.16M multiply-adds and 0.995 for a clocked conv net at 3.07M.
- **Depth is learned natively when credit is right (§83–§86).** Summed potentials with conserved multiplicative credit, credit to one instant with cooled exploration, and synapses grown only when credited (provably the same decisions as dense weights) learn order among three and four parts: 0.999–1.000 on 5/5 seeds at depth 4 with $\approx 2,000$ updates and $\approx 80k$ grown synapses out of $5.5\cdot 10^7$ candidates. With learned timing windows and credit to the latest instant, the composition task a Transformer led reaches 0.990–0.999 (mean 0.9965) from 40k examples seen once, vs 0.9955–0.998 for a Transformer after 2M (§88–§89).
- **Grokking occurs, by a route change under sleep, and only for relations the substrate can express.** Without sleep the network memorizes; with sleep it generalizes after a delay (0.93–0.99, reliable with cooled timing noise); a data $\times$ sleep phase diagram shows memorization, grokking and collapse; with depth, $(a + b + c) \bmod p$ through two composed rhythm stages reaches 0.99–1.00 (3 seeds, $p = 17$ and 31).
- **Learning cost follows activity, not model size:** 12 $\rightarrow$ 96 input channels leaves learning mistakes flat and makes inference cheaper (§77).
- **Not yet: real asynchronous benchmarks.** SHD below dense baselines; on the market, correctly posed as trading with costs, no learner profits and the native one learns to stay out; the online world model, built as an event network, beats a neural point process by 0.5–0.9 nats per event, also on held-out days.

Where the event paradigm wins, and where it does not

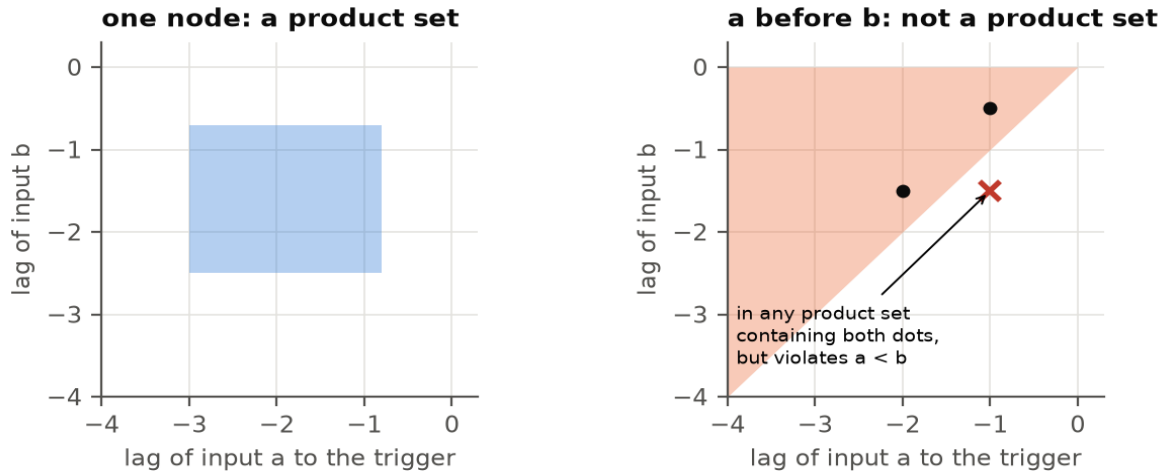
“Supremacy” here means a measured advantage over dense networks given the same data; each row states its caveat.

claim	evidence	caveat
Equal or better accuracy at 10^4-$10^5\times$ lower cost on timing tasks	E35: 1.000 at 7.5 synaptic events, nothing given; conv net 0.995 at 3.07M MACs; event-token Transformer 0.989-0.996 at 146k-1.16M after $10\times$ more training	one task family; much of the gap is the clock (an event-driven conv net $\approx 10\times$)
Groks composed arithmetic where a Transformer does not	E41, $(a + b + c) \bmod 17$, 30% of triples: 0.994-0.999 (3/3) in 200 epochs; Transformer (AdamW, weight decay, 100k steps): 0.29 / 0.63 (seed 0, $d = 32 / 64$), 0.06 / 0.03 (seed 1; chance 0.06)	the two-stage rhythm route is a provided resource (E45: it can choose among routes); more steps might help the Transformer
World model: beats a GRU; within 0.07-0.18 nats of a Transformer point process at $\approx 1/3000$ of its cost	E48: online -2.11 vs GRU -2.62 nats/event; held-out frozen $-2.38 / -2.10$ vs $-3.15 / -2.98$; ≈ 19 synaptic ops vs thousands of MACs	the Transformer Hawkes process is more accurate (held-out $-1.97 / -1.82$ vs $-2.15 / -1.96$, same hazard family)
Learning cost follows activity, not model size	E35: $12 \rightarrow 96$ channels, 0.999-1.000, mistakes flat, inference cheaper ($7.5 \rightarrow 3.2$ -4.0 synaptic events); \$77, \$81	measured to 96 channels
Structure discovery, implicit Occam razor	E45 pilot: one rhythm for $a + b$ (1.000), the chain for $a + b + c$ (0.999), nothing for random tables	pilot
Deep order learned from few examples	E54: 20 orders of four motifs, 0.999-1.000 (5/5) after 10-15k examples, $\approx 2,000$ updates, ≈ 150 events, $80k$ of $5.5 \cdot 10^7$ candidate synapses grown	depth 3: a Transformer matches (0.996-0.999) at $\approx 5,000\times$ the computation; depth 4: 0.990 at equal data, 0.992-0.996 with 2M (4 - $10\times$ the error rate)
Composition: Transformer-level accuracy from one pass	learned windows + latest-instant credit 0.990-0.999 (mean 0.9965) from 40k examples once (E89); Transformer 0.9955-0.998 after 2M, 0.9935-0.9965 given the same $40k \times 50$	one seed at 0.990; dips without a margin
<i>Not supremacy: spoken digits (SHD)</i>	E59 class-conditional event world models, speaker-relative bands, selected on held-out speakers: 0.675 test (E51 0.647); LSTM ≈ 0.70 ; state of the art ≈ 0.9	unseen test speakers
<i>Not supremacy: trading profit</i>	E42 (21 unseen days): no learner beats buy-and-hold (+932 bp); the priced native one +226 bp, others lose	the data may hold no edge

1. What an event node computes

Primitives. Spike times; delay; first-of (min) and all-of (max); hold (an input opens a window of given duration); veto (an input blocks the node while within its window). A node fires at its trigger if its holds are met and no veto is active.

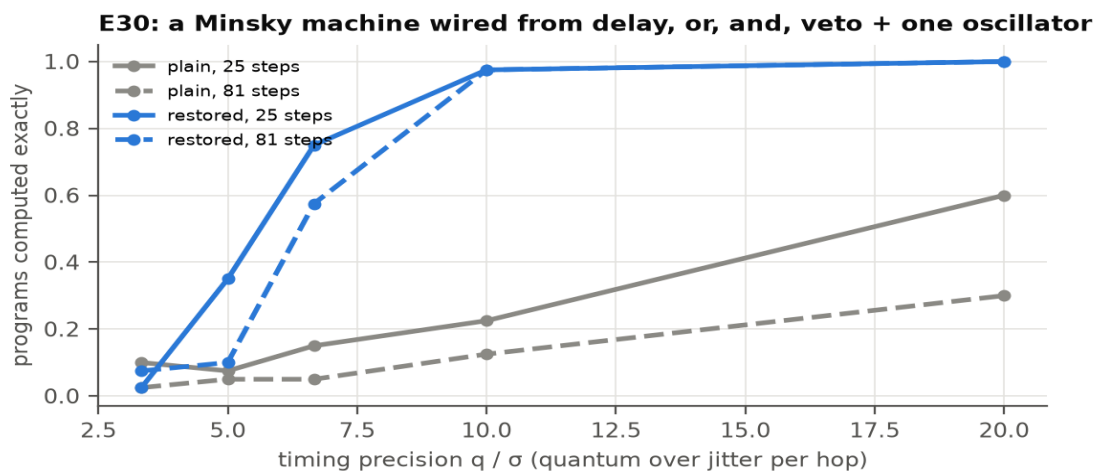
Theorem (§71). In lag coordinates relative to the trigger a stateless node accepts a product set. So one node orders at most three events (trigger the middle one, hold the first, veto the last until it arrives); two layers compute every conjunction of bounded differences with exclusions (every zone of timed-automata verification); three layers every finite union. *Evidence:* an exhaustive search over single nodes finds the proof's own node for 3 events and none for 4 on grids of 1,680 and 11,880 configurations; all thirteen Allen interval relations built at depth 2 are exact on ~19,900 interval pairs (E39).



Stateful nodes are stronger. A node armed by A, disarmed by C and fired by B computes an XNOR of two order relations, which no stateless node computes (exhaustive search, E39b); a toggle node computes parity.

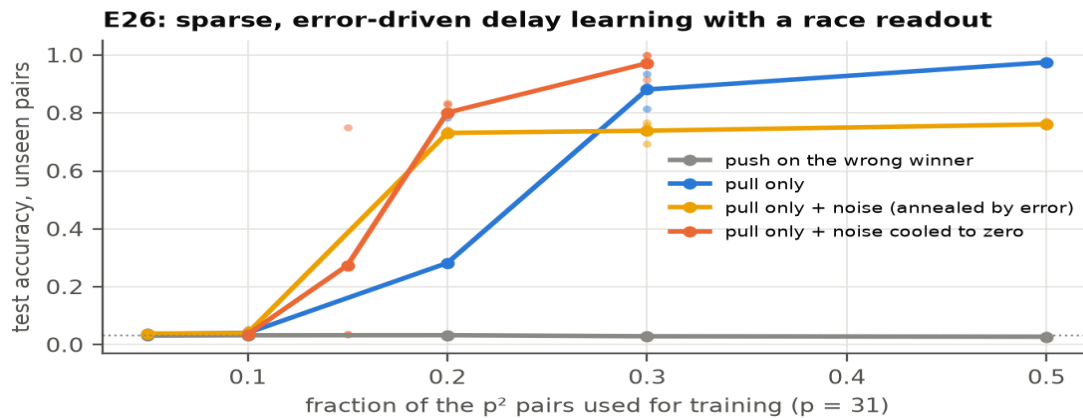
Clockless computation computes relations, not sums: shift symmetry forbids adding two times; one shared rhythm makes cyclic arithmetic computable (§56).

Completeness and reliability (E30). A two-counter machine wired from these primitives plus one oscillator runs exactly (Turing-complete given timing precision). Jitter accumulates like a random walk; one restoring coincidence per cycle makes reliability independent of program length.



2. How event networks learn

Winning is positional (§54). Pull the event that should have won; never push a loser later, which hands the win to the next loser and collapses the detectors. With learned delays on $(a + b) \bmod p$: push at chance in all 15 runs, pull-only 0.97–0.98 (E26). Timing noise cooled to zero makes small-data generalization reliable.



Order needs held intervals, not aligned delays (§61). Delaying A to meet B compresses the interval so a veto cannot see where C fell; A opening a window that B must hit keeps it. With holding, the detector for “B within Δ after A unless C” reaches 1.000 on 5 seeds with ~450 updates in 200k episodes, and veto is worth 19 points (alignment: 0.914, veto worth 1.8).

The rules are online learning on the simplex (§68): pulls moving a fixed fraction of a conserved budget are Winnow/Hedge-type updates with $O(k \log N)$ mistake bounds; durations are interval learning, vetoes monotone disjunctions. With nothing given, detectors learn channels, durations and vetoes: 1.000×4 , 0.9995, 443–1,530 updates (E35).

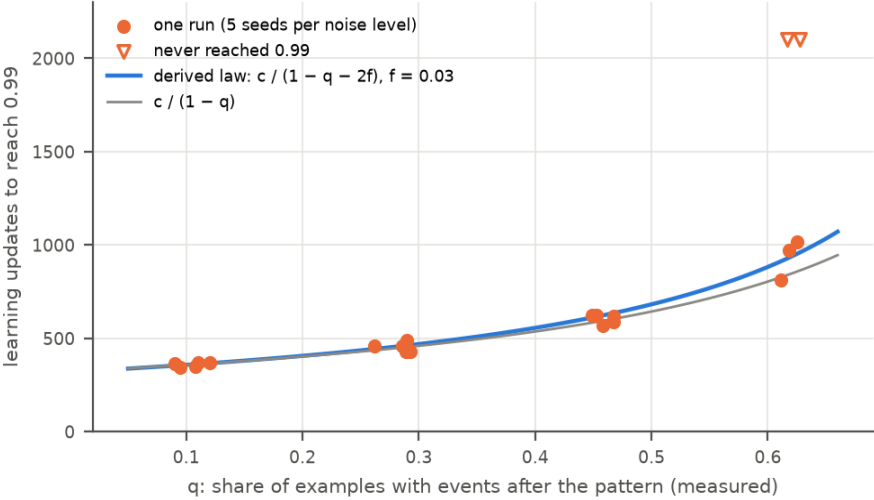
Routing needs counterfactuals (§57). Credit along a spike's causal path cannot say whether another route should have been taken; cancelled near misses supply it. Without it a two-layer network stays at chance (0.17 vs 0.72–0.74, E28).

A learning calculus for deep event networks (§83–§88). Learning deep order natively needed five rules, each derived from a failure, each local to a node and paid for by events:

problem	rule	why it works	evidence
an AND fails: which half was wrong?	sum inputs; multiplicative credit under a conserved budget	conservation moves weight toward the target on every false fire iff $\theta > \frac{1}{2}$ budget (§83, proved)	$4 \times$ candidates $\rightarrow 1.3 \times$ updates (E34w)
candidates fire at different instants	credit one instant, chosen by cooled exploration	all-instant credit deadlocks (proved); greedy cycles on shared prefixes; valid routes absorb (§84)	depth 3: 0.998–0.999 vs 0.71–0.85 greedy, 0.33 all-instant (E53)
the candidate basis grows as P^{depth}	grow a synapse when first credited	exactly the decisions of dense weights (§85, proved, checked)	depth 4: 80k of $5.5 \cdot 10^7$ grown, 0.999–1.000 (E54)
converged nodes sit on their threshold	near-miss margin, earned by recent precision	margins survive r demotions (§86); unearned margins protect wrong routes (§86b)	0.997–0.999 at every checkpoint (E53g)
timing precision	tune one window per part from its own lags	a window bank costs activity quadratic in resolution; tuning costs none (§88)	learned = hand-tuned windows (E56)
first-to-fire commits on partial evidence	credit the latest candidate instant on a miss	the pattern is complete only at its last event; earlier instants are prefixes (§89)	composition 0.990–0.999; depth 3 at half the updates (E89)

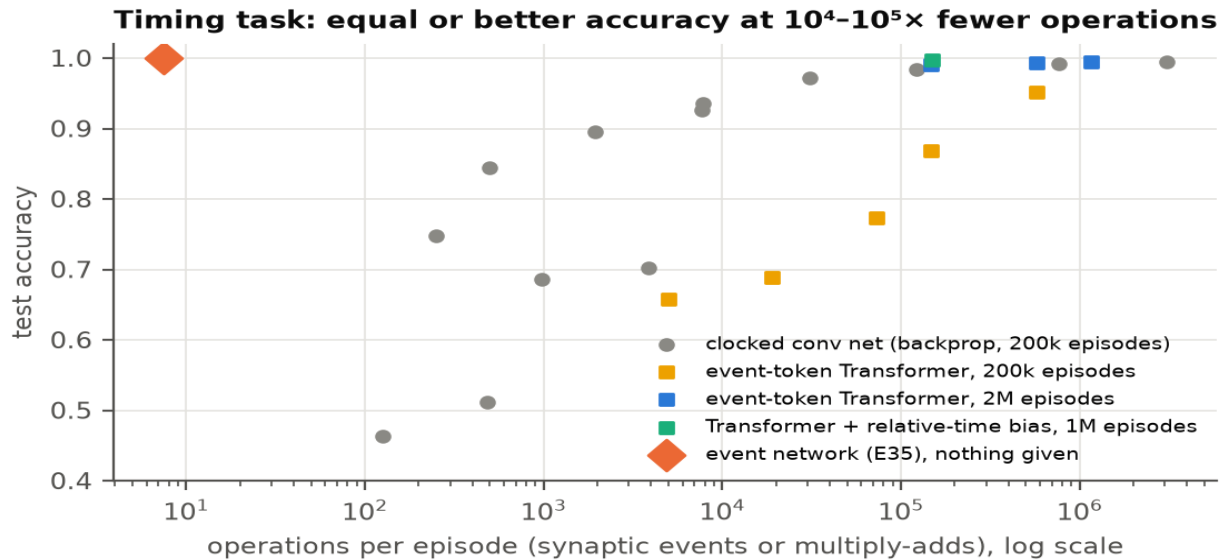
A derived law, measured (§91). Latest-instant credit works because noise after the pattern is inconsistent between examples; the bound says learning slows as $1/(1 - q - 2f)$, q the share of examples whose last event is noise. Sweeping the noise and measuring q , the updates needed follow the law with one fitted parameter ($f \approx 0.03$):

Crediting the latest instant: learning slows as $1/(1 - q - 2f)$, as derived (§91)



3. Against dense models and Transformers

model	test accuracy	cost per episode	learning
event network, nothing given (E35)	1.000 ×4, 0.9995	7.5 synaptic events	443-1,530 updates, 200k episodes
event-token Transformer, 2M episodes	0.989-0.996	146k-1.16M MACs	backprop
Transformer + relative-time bias, 1M episodes	0.996, 0.998	≈ 150k MACs	backprop
event-token Transformer, 200k episodes	0.65-0.97	5k-576k MACs	backprop
clocked conv net (E32)	0.995 / 0.984 / 0.895	3.07M / 123k / 1.9k MACs	backprop, 200k

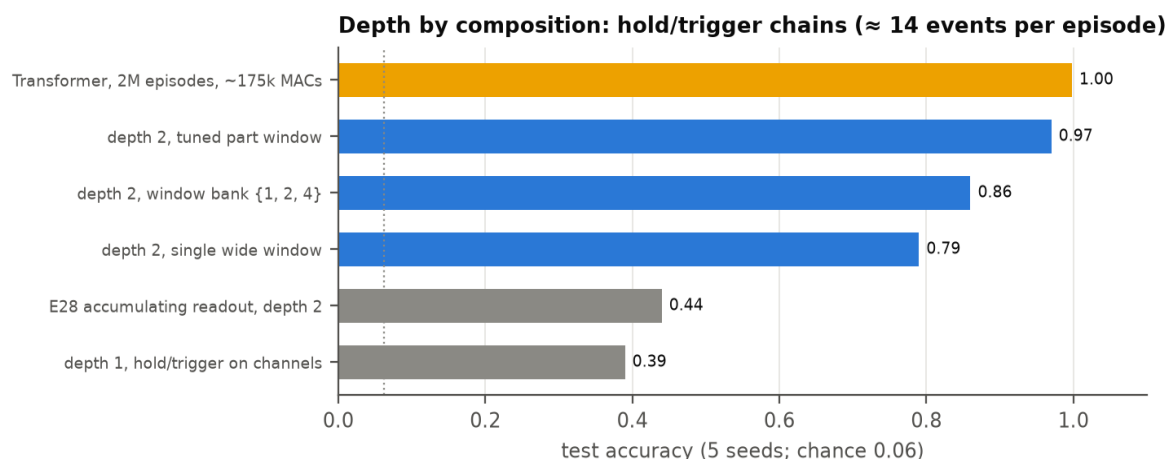


Scaling with the input basis (E35, §77). At fixed spikes per episode, $12 \rightarrow 96$ candidate channels leaves accuracy ≈ 1.0 (96: 1.000, 0.999, 1.000) and learning updates flat while synaptic events per episode fall: learning and inference cost follow activity, not the size of the basis.

Why. Attention cannot see order without position information and must synthesize time comparisons from dot products at $O(n^2 \cdot d)$ per layer; a hold/trigger node computes the comparison as its primitive, at the cost of its input events (§70). The clocked model also pays per time bin, so silence multiplies its cost; a conv net evaluated only where spikes are would cost ≈ 100 multiply-adds, not millions (§63). At published per-operation energies the gap at equal accuracy is $\approx 10^4\times$. *Limits:* one task built around the primitives; the Transformer needed $10\times$ more training for parity; with a learned relative-time attention bias (time differences enter attention directly) it reaches 0.996-0.998 at $\approx 150k$ MACs, still below the event network at $\approx 2 \cdot 10^4\times$ its cost.

4. Depth and composition

Classes that are ordered combinations of shared parts are zones over part events, so the theorem prescribes two layers: parts (hold nodes on channel pairs) and class nodes that hold one part's spike and are triggered by another's, so a single part cannot satisfy them. Learned natively: 0.97 with part windows at the motif scale, 0.86 with a generic bank of window scales, depth 1 0.39 (5 seeds, E34). A readout that accumulates evidence fails at conjunction even though its hidden nodes learn parts (E28). An event-token Transformer trained on 2M episodes reaches 0.998 at $\approx 175\text{k}$ – 690k multiply-adds vs ≈ 14 events. The chains are not training-limited: at 200k episodes they stay at 0.954 (tuned) and 0.87 (bank); some class routes lock onto wrong parts.



What the credit rule must be (§83–§84). A class node should sum its held and its coincident trigger inputs and learn by full-information multiplicative updates under a conserved budget. Proved: conservation resolves the AND's credit ambiguity (a false fire does not say which half was wrong) iff the threshold exceeds half the budget; the mistake bound grows with the log of the candidate basis ($16 \rightarrow 32$ channels: $1.3\times$ the updates). Crediting every candidate deadlocks when units fire in every positive at different instants (window bank: 0.05–0.10); crediting the one instant closest to firing can cycle between invalid prefixes; cooled exploration over instants reaches the valid route, which is absorbing. With a class window covering the task's span, E34's task plateaus at 0.988–0.995 per seed and the generic bank at 0.97–0.99 with fixed windows. **Learned windows (§88) plus credit to the latest instant (§89)** remove the last errors, lost races to a shortcut that fires when the second motif merely begins: 0.990–0.999 (mean 0.9965) from 40k examples seen once, against a Transformer's 0.9955–0.998 after 2M examples and 0.9935–0.9965 given the same 40k examples 50 times: parity at equal data, from one pass, at $\approx 10^4\times$ less computation.

Order among three parts: depth 3 (E53, E54). Classes that are different orders of the same motif sets (20 classes, decoys = other orders) need ordered intermediates: composite units $u \rightarrow v$ over every ordered pair of parts (57,840 candidates, ≈ 41 events per episode). 5 seeds, 40k episodes:

E53	test accuracy	updates
depth 3, + margin earned by reliability (§86b)	0.997–0.999 at every checkpoint, 5/5 seeds	$\approx 1,100$ – $1,400$
depth 3, instant credit, exploration $T = 0.3$	0.998–0.999 on all seeds by 5k episodes; final 0.963–0.999 (dips)	$\approx 1,100$
depth 3, greedy instant credit	0.71–0.85 (cycles between shared prefixes)	4k–11k
depth 3, credit to every candidate	0.33–0.36 (never fires)	$\approx 26\text{k}$
depth 2, same credit	0.32–0.42	$\approx 24\text{k}$
Transformer, 2M fresh examples	0.997–0.999	215k–843k MACs/example
Transformer, 40k $\times$ 50 passes (wd 0 / 0.1)	0.9975, 0.996 / 0.981, 0.994	215k MACs/example

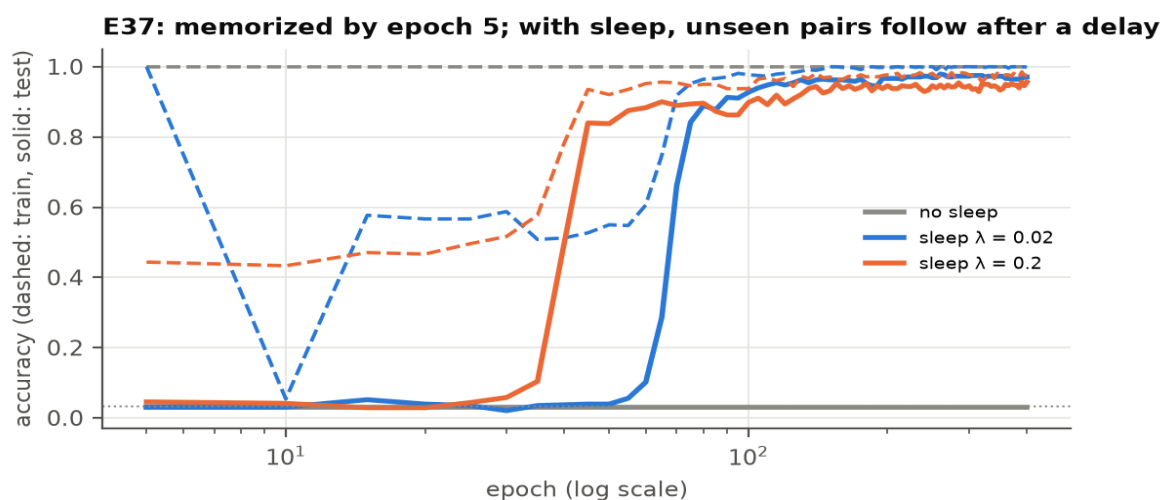
Without a margin, converged classes dip (a node sits just above threshold; one demotion knocks it under). A margin kept by near-miss credit removes the dips but also protects wrong routes (3/5 seeds

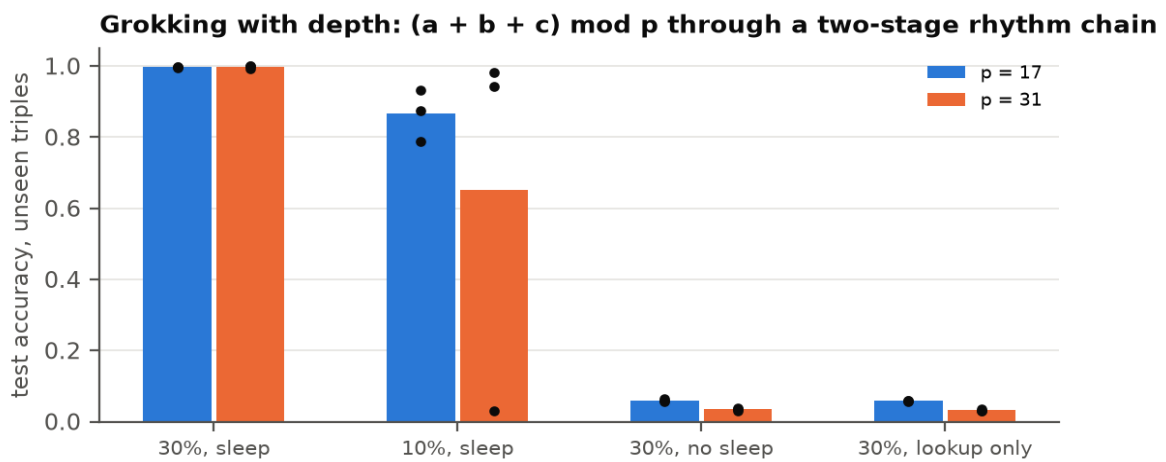
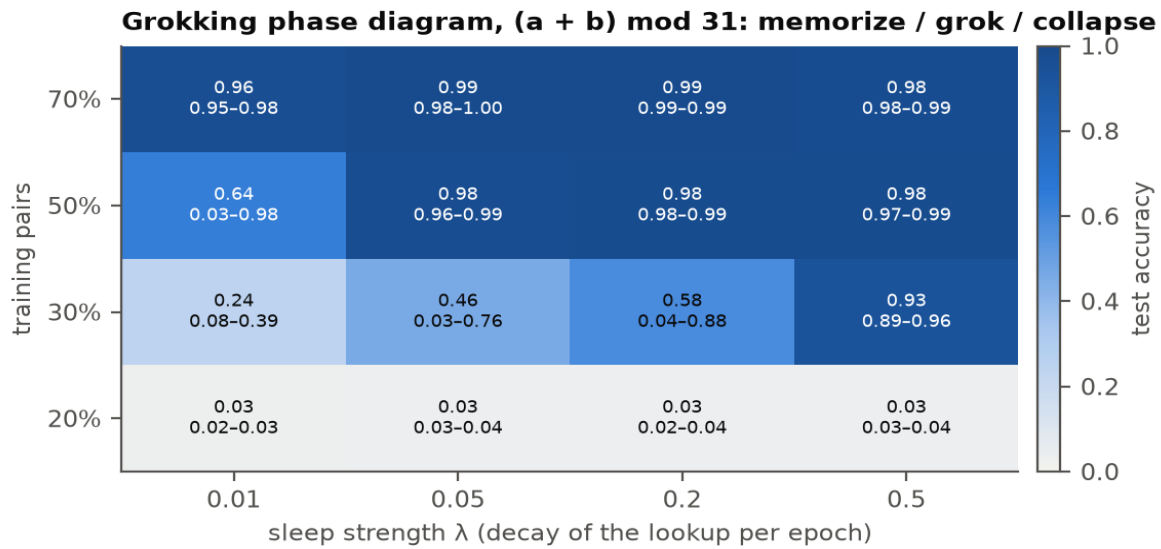
freeze on a shared prefix); a margin earned by reliability, applied only when the node's recent fires were mostly correct, is stable at 0.997–0.999 (§86b). With synapses grown only when first credited (§85, provably the same decisions as dense weights), depth 3 at 20 channels gives 0.981–1.000 with 9.5k–13.7k of 145k candidate synapses ever grown. **Inference cost follows the learned structure (§93):** extending a unit only if one of its children carries weight (checked periodically, like sleep) keeps accuracy (depth 3 identical; depth 4 0.995–1.000) while events per example fall 42% at depth 3 and 75% at depth 4 ($\approx 155 \rightarrow 39$).

5. Generalization and grokking

What counts (§58): restriction, forced generalization above capacity, and grokking (the relation reached while memorizers are available) are different claims; each reports $\rho = n/\text{params}$. **Per-class parameters cannot generalize on $(a + b) \bmod p$ (§66):** each operand occurs once per class, so generalization needs shared intermediates; in time, a sum needs a rhythm.

Grokking as a route change (E37). Each class has a pair-node lookup that can memorize everything ($\rho \approx 0.016$) and a shared route through a rhythm with learned delays; learning is errors-only. Without sleep: train 1.0, test 0.03–0.04. With sleep: test 0.93–0.97 in 2 of 3 seeds without timing noise, and 0.95–0.99 in 5 of 5 seeds with cooled timing noise on the shared route ($\sigma = 2$), after a delay. Sleep without the rhythm: train 0.45, test 0. The rhythm is used only where it fits: $a - b$ and relabelled sums grok (0.95–0.98), $a \cdot b$ in 1 of 3 seeds, while $a^2 + ab + b^2$ and random tables stay at chance on unseen pairs (0.01–0.04) and their training accuracy erodes under sleep. A data $\times$ sleep phase diagram (4×4 , 3 seeds) shows no grokking below 20–30% of pairs, and above it a minimum sleep that falls with data. Larger problems grok more reliably: 2/3 seeds at $p = 31$ and 59, 3/3 at $p = 97$ ($\rho \approx 0.005$). With depth (E41, 3 seeds): $(a + b + c) \bmod p$ through two composed rhythm stages, 0.99–1.00 on unseen triples from 30% of them at $p = 17$ and 31, and from 10% in 2 of 3 seeds at $p = 31$ ($\rho \approx 0.003$); chance without sleep; collapse without the chain. The data it needs is far above the Occam bound (§78): at 1–7% of triples it memorizes and stays at chance, also when trained 7–10 $\times$ longer; the threshold at $p = 17$ lies between 7% and 30%.



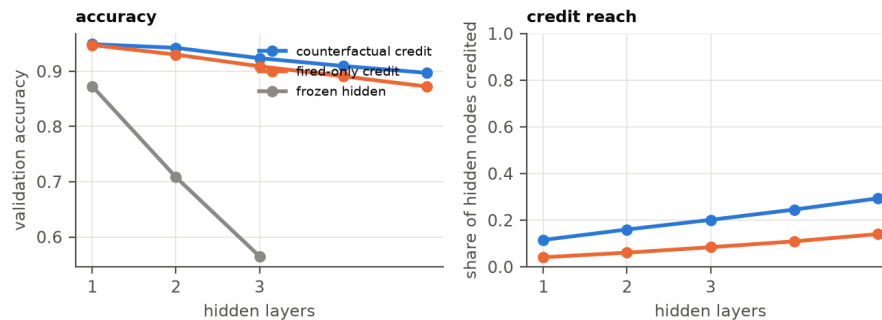


Why (§69, §72). Error-gated learning makes memorization absorbing. Sleep keeps a parameter only if it is used by more than $m^* = \lambda\theta/(e\eta)$ examples: lookup entries serve one and die, the rhythm's delays serve many and survive, and once the rhythm answers a pair its lookup entry is never relearned. This predicts memorization, grokking and collapse regimes (all observed) and a data threshold $n^* \propto p\lambda\theta/(e\eta)$.

6. The weight race

The original architecture (integrate-to-threshold nodes with learned weights, racing in groups) holds its first results: a cancelled node can be taught (E4: 0.79 at $K = 128$ where reward-modulated rules are at chance); races decide as fast as the evidence allows (E2, matching the optimal MSPRT); learning work tracks activity (E5: $\approx 300\times$ fewer updates); local learning reaches ≈ 0.96 on latency-coded MNIST with counterfactual credit that pays more with depth (E6, E14).

E14 · the advantage of counterfactual credit grows with depth (2 seeds)



Its limits: depth still costs accuracy; only its single racing layer beats an equal dense model at inference ($\approx 2.4\times$); it forgets more than SGD (E23); it memorizes instead of grokking (E24); and the rules that fixed the timing networks do not transfer to its weights (SHD 0.04–0.29 vs 0.35).

7. Real data

- **Spiking Heidelberg Digits** (spoken digits as cochlear spike trains, 700 channels, 20 classes, unseen test speakers): class-conditional event world models (E51; state = last spike's band, time since it, time since onset; one counting pass) reach 0.647 test (0.734 on held-in speakers); timing +0.06, onset reference +0.21. The weight race reached 0.35; a published LSTM ≈ 0.70 ; state of the art ≈ 0.9 . SHD is only $\approx 6\times$ sparser than a 10 ms raster, a weak test of the paradigm's cost advantage. Validating on held-out speakers and coding bands relative to each voice (a running centroid per utterance) raises held-out-speaker accuracy from 0.36–0.38 to 0.44–0.46 and the test to 0.675 (E59, §92).
- **Market stream posed as trading with costs (E42, confirmed on 21 unseen days, preregistered)**: at 2 bp, imitating a hindsight teacher over-trades and loses (event learner –15,236 bp, logistic –24,302); the profit-priced event learner nets +226 bp with 8 changes; buy-and-hold +932; at 10 bp all stay out. No learner beats buy-and-hold; pricing the decision is what stops the losses.
- **Is staying out right? An edge audit (E55, §87)**. From executable round trips on the tape (buy at the ask, sell at the bid), BTC spot's own event states carry a real held-out edge of +0.3 to +0.9 bp per trade before fees, and lead-lag states of BTC perpetual futures and ETH raise it to +1.1 to +1.5 bp: more markets carry more information. A 2 bp round-trip fee (a tenth of a realistic taker fee) removes it: staying out is correct for a taker here; the edge would need market-making economics. **Confirmed on the 21 untouched days** (preregistered): before fees +0.26 to +1.05 bp from the own state, +0.47 to +1.40 bp with perp and ETH; at 2 bp every selected state loses; at 5 bp none qualifies. **Across four markets** (ETH spot, SOL spot or the BTC perpetual traded, the others as leaders; choices made on the pilot days): at most ≈ 1 bp before fees, nothing at 2 bp (E55b).
- **Event-camera gestures (DVS128 Gesture, E60; official split)**. Native motion events (refractory cells, onsets, direction-selective pair detectors with an opponent veto) turn $\approx 410k$ raw events per gesture into $\approx 15k$. A bag of motion events reaches 0.663 on unseen people; depth (successive motions per region) with the multiplicative learner reaches 0.776 on held-out training users and 0.701 on the test users (chosen on validation). Published systems reach 94–98%: not yet competitive; the representation misses rotation sense and trajectory shape.
- **Against a Transformer point process (E52, E57, §90)**. A Transformer Hawkes process with the event network's own hazard family and 128 events of context, selected on day 5, scores –1.97 / –1.82 nats per event on the held-out days ($\approx 108k$ multiply-adds per event; –1.83 / –1.65 with 12

finer windows). The semi-Markov event network scores $-2.38 / -2.10$; slow regime state (leaky event counters at 5 s and 60 s, an order-flow counter, backoff) brings it to $-2.18 / -2.00$ (fine windows $-1.93 / -1.75$) at ≈ 30 operations per event. The Transformer is the better world model by 0.07–0.18 nats; the event network gets within that at $\approx 1/3000$ of the computation. Counted slow state transfers to unseen days; constant-step multiplicative factors track the end of training and do not (E58).

- **The world model is an event network, and it beats a neural point process (E44, E48).** A likelihood decomposition located the GRU's lead in which event comes next; count baselines located the missing information (the time since the last event). A semi-Markov event network (state nodes for the last two types, window nodes from a delay line, count-learned detectors) reproduces that model exactly and scores -2.11 nats per event online (days 1–5) and $-2.38 / -2.10$ frozen on held-out days 6 / 7, against -2.62 and $-3.15 / -2.98$ for the GRU, at ≈ 19 synaptic operations per event. The model class is classical; an offline-trained GRU is the remaining check.
- **Earlier native world model (E44), prequential log-likelihood per event (nats; days 1 / 2 / 3):** Poisson $-3.00 / -3.42 / -3.32$; Hawkes (Adam) $-2.62 / -2.94 / -2.84$; native (multiplicative) $-2.64 / -2.85 / -2.70$; GRU neural point process $- / -2.61 / -2.52$. Pair-part state neutral; learned inhibition below excitation-only.
- **Online world model (E44):** a temporal point process of four event types learned from every event; the native model ($-2.64/-2.85/-2.70$ nats per event) matches Hawkes ($-2.62/-2.94/-2.84$) and trails a GRU neural point process ($-2.61/-2.52$) by ≈ 0.2 nats.

8. Open problems and next steps

- **Stability of the full rule set on every task at once:** the margin earned by reliability is stable at depth 3–4 and with fixed windows but hurts when windows are learned (it entrenches early shortcuts at the firing instant and promotes trailing noise at the latest instant, §89): the margin needs another anchor.
- **Depth beyond four and denser streams:** depth costs activity $n \cdot r^L$ (§85); demand-driven propagation is the untested remedy; dense streams (spoken digits, §55) are where it matters.
- **Structure discovery for grokking** (E45 pilot picks correctly) and **the data threshold of grokking** (7–30% of triples, far above the Occam bound; the sleep reuse filter is the candidate constraint).
- **Transformer baselines** on the depth-3/4 tasks and a Transformer Hawkes process on the market stream (running).
- **Native learning of sparse parity** (§75) and **a real benchmark with rare, precisely timed events** (§55).
- **Joules, not operation counts:** run trained networks on neuromorphic hardware (§9).

9. Hardware: what these networks need, and what exists

Each primitive the theory settled on maps to a hardware feature:

what the network does	what hardware must provide	why
work only when an event arrives	event-driven execution; memory next to compute	cost follows activity (§55, §77)
delays and hold windows	per-synapse programmable delays, per-node hold timers, timestamps	order is held intervals (§61, §71)
first to fire wins, the rest cancelled	fast arrival-order resolution and inhibition	the race readout (§59)
multiplicative credit under a conserved budget	per-synapse multiply, per-node renormalize, contribution tags, local random source, precision counter	§83–§86b, §89
synapses grown when first credited	run-time allocation in a sparse synapse store	basis 10^5 – 10^{10} , 10^4 – 10^5 grown (§85)
part windows tuned from their own lags	per-synapse window edges with a local rule	§88

The optimal machine (a sketch). Clockless digital cores with timestamped events, per-synapse delay and window fields and per-node timers; arrival-order comparators and inhibition trees for the race; a small event-triggered learning engine per core; a content-addressed sparse synapse store with allocation on credit. This is close to *race logic* (first arrival = minimum, delay = addition: the max-plus algebra of §79) plus learning. At ≈ 24 pJ per synaptic event (Loihi, 2018) these networks would spend ≈ 0.2 nJ per example on the timing task, ≈ 0.5 nJ on composition and ≈ 4 nJ at depth 4; a Transformer at 175k multiply-adds per example on a GPU is on the order of a microjoule (orders of magnitude, not measurements).

What exists (September 2026):

system	availability	fit
Intel Loihi 2 / Hala Point	research access; Hala Point a prototype (Sandia); Loihi 3 announced, no public specs	best for prototyping: event-driven; weight, delay (≤ 62 steps), tag per synapse; microcode learning rules. Missing: run-time synapse allocation, long delays, timestamps
SpiNNaker2 (SpiNNcloud)	commercial systems; 152 ARM cores/chip	most flexible: every rule incl. synapse growth in software; less efficient per event; time-stepped
BrainChip Akida / Akida Pico	commercial; Pico in FPGA-cloud evaluation (2026)	converted CNN-style SNNs, limited learning, no suitable delays: poor fit
Innatera Pulsar	volume production (2026), μW -mW	deploy small trained networks at the sensor
SynSense Speck / Xylo	commercial dev kits	inference-only spiking ASICs for vision / audio
DYNAP-SE2, BrainScaleS-2	research	analog continuous time; device mismatch
IBM NorthPole	research	synchronous dense inference: not a fit
FPGAs	commercial	delays as timestamp queues, synapses in BRAM hash tables: the most faithful full implementation today
event sensors (Sony/Prophesee, iniVation)	commercial	natural front ends: they emit these event streams

What to do with it. Measure joules: trained networks on Loihi 2, the full learning calculus on SpiNNaker2 or an FPGA, against a Transformer on a GPU; deploy frozen networks on sensor-edge chips. The feature no commercial chip offers, and the one these results say matters most, is **run-time synapse allocation on credit** with per-node conserved budgets: it lets a network search 10^7 – 10^{10} candidates while storing only what it uses. Sources are listed in REPORT.md §9.

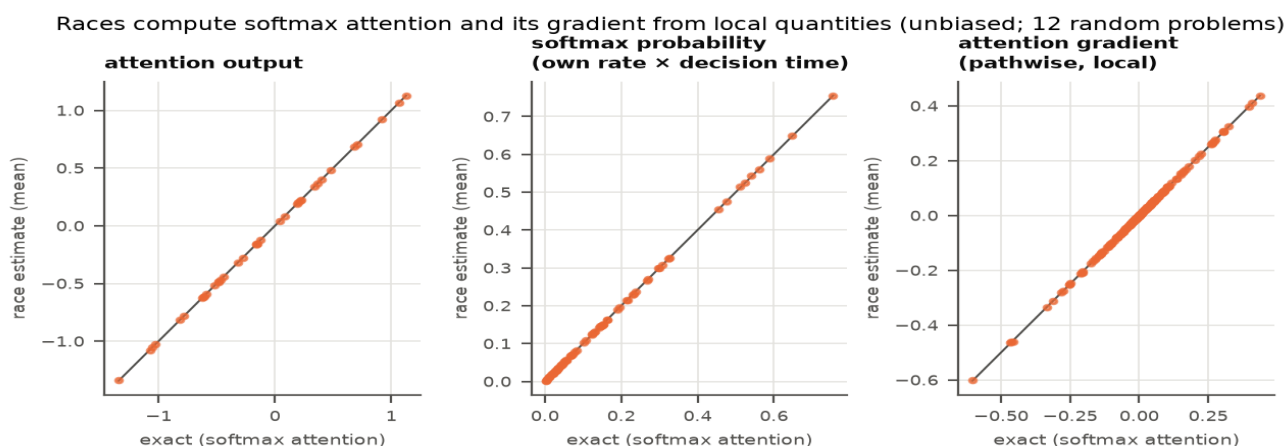
10. Next frontier: generative language models

The aim is not to approximate Transformers but to exceed them: the same or better quality, with work per word that does not grow with model size or text length, learned by local rules from less data.

Why that is a reasonable aim (theory, §96–§98):

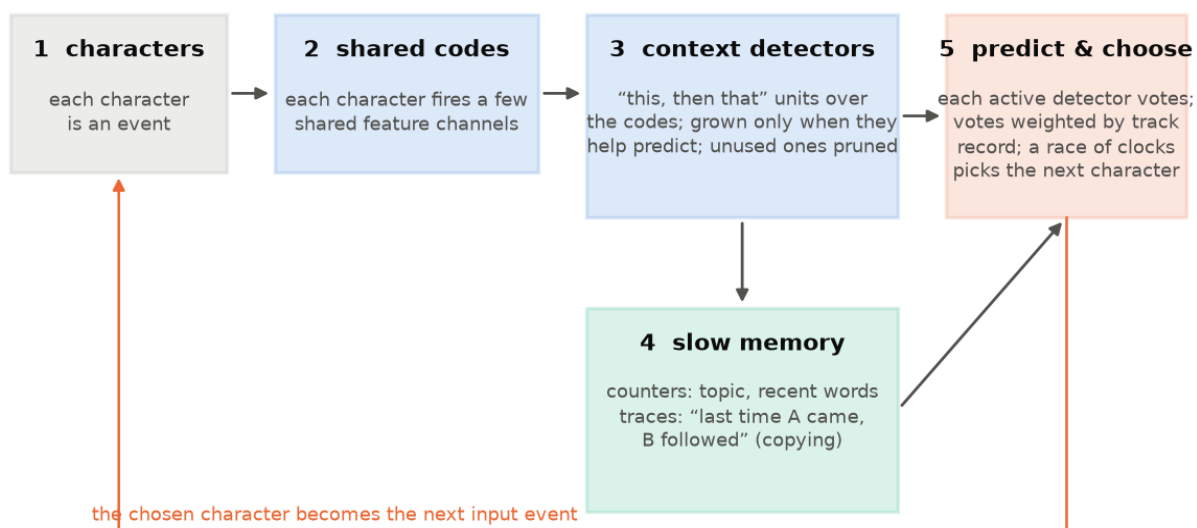
- **Nothing a Transformer computes is out of reach.** Query-key similarity is the overlap of spike codes; a race among stored keys picks the best match, and a race of randomly ticking clocks picks each key with exactly its softmax-attention probability; relative position is native; the feed-forward block is threshold units over codes; stacking layers is composition. A Sleeping Machines network can express any Transformer.
- **It has freedoms a Transformer lacks:** the order in which signals fire carries up to $\log_2 n!$ extra bits for n signals; only active units work, so a model can be very large while each word stays cheap; structure grows where it proves useful; sampling is a race; retrieval reaches only keys sharing a channel with the query.
- **Local learning is not a handicap in principle.** A race computes with minima and sums; the exact gradient backpropagation would compute runs only along the chain of spikes that caused the output, which each node traces locally: credit along that chain is backpropagation for these networks, and near misses supply the signal gradients cannot give to losing paths. For races of random clocks the exact gradient is local too.
- **Depth is trainable, optimally.** For ordered-pattern detectors of depth d , mistakes grow as $d \times \log(\text{candidate pool})$, and no learner can do better in the worst case (§97–§98).

Training, subsumed (theory, §101-§103). A race of randomly ticking clocks splits its output into two independent channels: which clock fires first (a sample from the softmax) and when (the decision time, which carries the softmax's normalizer). Proved: every competitor computes its own softmax probability locally (its rate times the decision time); keys emitting values scaled by that product give, on average, exactly softmax attention; and the locally computed gradient of this race is, on average, exactly the gradient of softmax attention. A network of such races with small dense cores is trained by local message passing as stochastic gradient descent on the Transformer objective, up to an error shrinking as $1/R$ with R races per head: Transformers, including their training, are a limit of these networks. Numerical check below; training curves compared in E68 (running).



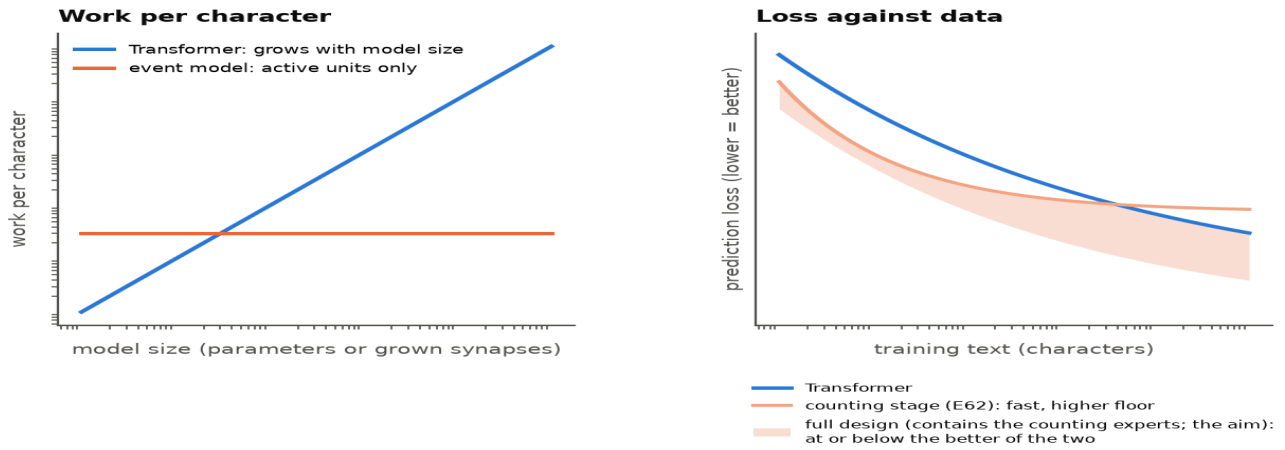
First evidence. Deep order is learned from about ten times less data than a Transformer needs (section 4). Attention is learnable by local credit: in a recall task where the network must learn which key a query refers to and which neighbour to read, a race-attention layer trained by local credit alone is 100% correct after 68 mistakes, and stays 100% on contexts four times longer than trained on (one run; five runs and a Transformer comparison running, E61).

A planned event language model: work only where characters arrive (design, not yet built)



The plan, in stages, on character-level text (text8): (1) a counting baseline with a copy memory (running, E62), not the goal but a measurement of how memory and loss scale with data; (2) race attention over the stream, learned by local credit; (3) learned shared codes; (4) stacked layers with credit along causal chains and near misses. At each stage: a recurrent network and a Transformer trained by gradients on the same text; bits per character, examples needed, work per character.

Predicted scaling (schematic, from §95–§96; not measured)



Established and not. Established: expressive equivalence, locality of exact credit for races, optimality of the depth bound, learned attention on a recall task, data efficiency on deep order. Not yet shown: that stacked race-attention layers with learned codes, trained by local credit, match or beat a Transformer on language itself; the stages decide it.

Every mechanism is an event handler (local state, triggered by events, cost proportional to events); dense procedures are diagnostics only. Reproduce: `python report/figures_time.py` & `python report/make_pdf.py`.